Heap

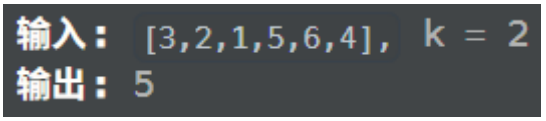
堆是一棵**完全二叉树**（Complete Binary Tree），这意味着除了最后一层外，其他层都是满的，且最后一层的节点都靠左排列。

- **大根堆 (Max Heap / 最大堆)**：
  - **定义**：任意节点的值都 **大于或等于** 其子节点的值。
  - **根节点特性**：**根节点是整个堆中最大的元素。**
  - **用途**：常用于需要快速获取最大值的场景（如 Top-K 大问题）。
- **小根堆 (Min Heap / 最小堆)**：
  - **定义**：任意节点的值都 **小于或等于** 其子节点的值。
  - **根节点特性**：**根节点是整个堆中最小的元素。**
  - **用途**：常用于需要快速获取最小值的场景（如 Dijkstra 算法、Prim 算法、Top-K 小问题）。

leetcode

215.数组中的第K个元素

- 整数数组 `nums` 和整数 `k`，请返回数组中第 `k` 个最大的元素。



- 思路：建立大根堆（**堆顶元素（根节点）永远是当前堆中的最大值**）。移除前k-1个元素，剩下的堆顶即为第k个大的元素。

```
class Solution {
public:
    void maxHeapify(vector<int>& a,int i,int heapSize){
        int largest = i;
        int l=i*2+1,r=i*2+2; // 结点i的左右两个孩子
        // 左孩子的值大于当前结点
        if( l<heapSize && a[l] > a[largest] ){
            largest = l;
        }
        // 右孩子的值大于当前结点
        if( r<heapSize && a[r] > a[largest] ){
            largest = r;
        }
        // 如果largest不为当前最大结点，则需要交换
        if( largest!=i ){
            swap(a[i],a[largest]);
            maxHeapify(a,largest,heapSize); // 递归处理交换下去的那个节点（它可能破坏了下一层的堆）
        }
    }

    void buildMaxHeap( vector<int>& nums,int heapSize ){
        // 从 最后一个非叶子节点 开始，向前遍历到根节点
        for( int i=heapSize/2-1; i>=0; i-- ){
            maxHeapify(nums,i,heapSize); // 对每个结点重新构建堆
```

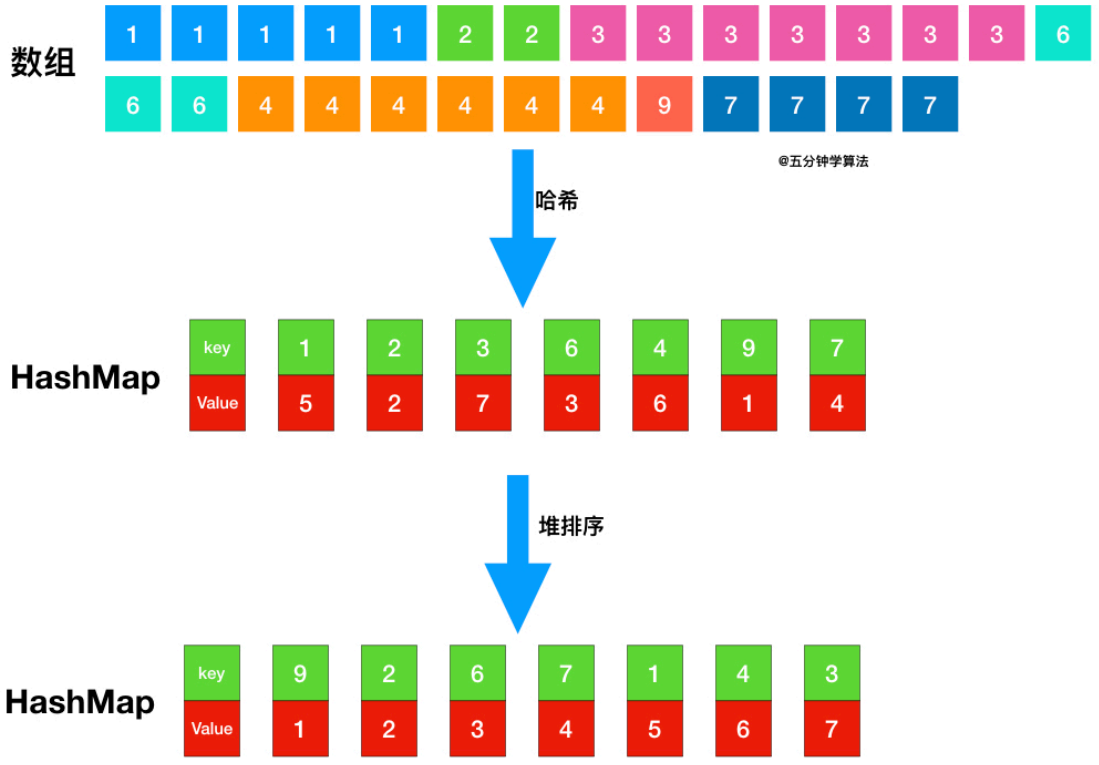
```
        }
    }

    int findKthLargest(vector<int>& nums, int k) {
        int heapSize = nums.size();
        // 构建大根堆
        buildMaxHeap(nums,heapSize);
        // 执行 k-1 次“删除”操作
        for(int i=nums.size() - 1; i>=nums.size()-k+1; i--){
            swap(nums[0],nums[i]); // 最大值被放到了数组末尾（相当于从堆中移除了）。
            heapSize--; // 缩小堆范围
            maxHeapify(nums,0,heapSize); // 重新构建堆
        }
        return nums[0];
    }
};
```

-

347.前K个高频元素

- 整数数组 `nums` 和一个整数 `k` ，返回其中出现频率前 `k` 高的元素。可以按 **任意顺序** 返回答案。
- 思路：**哈希表统计 + 小根堆维护 Top K**。通过维护一个容量为 k 的**小根堆**，确保堆中始终保留当前遍历过的元素中**频率最高**的 k个元素。

- 借助 **哈希表** 来建立数字和其出现次数的映射，遍历一遍数组统计元素的频率
- 维护一个元素数目为 k 的最小堆
- 每次都将新的元素与堆顶元素（堆中频率最小的元素）进行比较
- 如果新的元素的频率比堆顶端的元素大，则弹出堆顶端的元素，将新的元素添加进堆中
- 最终，堆中的 k 个元素即为前 k 个高频元素



```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        // 使用哈希表统计每个元素出现的频次
        unordered_map<int,int> numsTofre;
        for( int num:nums ){
            numsTofre[num]++;
        }

        // 构建小根堆
        auto cmp =[&numsTofre](int a,int b){  // 定义比较器
            return numsTofre[a] > numsTofre[b];
        };
        // priority_queue模板参数- 堆中存储的数据类型、底层容器（默认vector）、比较器的类型
        priority_queue<int,vector<int>,decltype(cmp)> minHeap(cmp); // 显式传入cmp

        // 遍历哈希表，维护一个大小为k的小根堆
        for( auto& pair: numsTofre ){
            int num = pair.first;
            if( minHeap.size() < k ){ // 堆未满，直接入堆
                minHeap.push(num); // 自动调整堆结构
            }else{
                // 如果当前元素的频率大于堆顶元素的频率（堆顶是目前 k 个中频率最小的）
                if( pair.second > numsTofre[minHeap.top()]){
                    minHeap.pop(); // 取出堆顶元素
                    minHeap.push(num); // 插入新元素
                }
            }
        }

        // 取出堆中的元素
        vector<int> res;
        while(!minHeap.empty()){
            res.push_back(minHeap.top());
            minHeap.pop();
        }

        return res;
    }
};
```

-